Estudi previ P-PRE4

Sèries temporals, InfluxDB, emmagatzematge calent i fred, i Grafana

Pau Vila Pérez

Tots els càlculs es fan amb el sistema que hi haurà a la P4:

- 5 dispositius: el node real (`node01`, AtomS3 amb un DHT22) i 4 de simulats des de Node-RED (`node02` a `node05`).
- 2 magnituds: temperatura (°C) i humitat relativa (%).
- 1 lectura cada 5 s per magnitud (el període que hi ha ara, retingut al broker).
- Gateway: Raspberry Pi 4 de 2 GB, amb Raspbian 13 de 32 bits i una targeta de 14 GB (9,1 GB lliures).

Això dona 10 sèries i 2 punts per segon: 17.280 punts al dia per sèrie i 172.800 en total.

Bloc A · Per què una base de sèries temporals

A1. Propietats de la telemetria

Comparada amb les dades d'una aplicació de gestió (per exemple, les comandes i els clients d'una botiga en línia), la telemetria té aquestes propietats:

- Només s'afegeix: una lectura no es modifica mai. En una aplicació de gestió les dades canvien constantment (l'estat d'una comanda, l'adreça d'un client) i la base de dades ha d'evitar que dues persones canviïn la mateixa dada alhora.
- Va ordenada en el temps: cada mostra porta la seva hora i arriba gairebé sempre en ordre. El temps és la dada principal.
- Arriben moltes dades i sense parar: el meu sistema escriu 2 punts per segon (172.800 al dia), de dia i de nit. Una aplicació de gestió té poques escriptures i de tant en tant.
- És molt repetitiva: dues lectures seguides del DHT22 són gairebé iguals i arriben cada 5 s, i per això es comprimeix molt bé.

A més, quan es consulta, sempre es demana un tros de temps (l'última hora, ahir) i normalment un resum (mitjanes, màxims), sobretot de les dades recents.

A2. Telemetria en una taula de PostgreSQL

Amb el meu sistema, una taula `(id, node, magnitud, valor, ts)` arribaria als 100 milions de files en uns 579 dies. Apareixerien aquests problemes:

- Ocupa molt: cada fila ocupa uns 60–70 bytes, i per consultar ràpid calen índexs, que també ocupen. Entre la taula i els índexs passarien dels 10 GB, més del que queda lliure a la targeta. En una base de sèries temporals, a 1–3 bytes per punt, serien uns 100–300 MB.
- Escriure és cada cop més lent: cada vegada que s'afegeix una fila també s'han d'actualitzar els índexs. Quan són tan grans que no caben a la RAM (2 GB al Raspberry), cada fila nova obliga a llegir i escriure a la targeta.

– Les consultes llargues són lentes: per fer la mitjana d'un mes s'han de llegir 5,2 milions de files senceres, perquè PostgreSQL guarda les dades per files i llegeix també les columnes que no necessita.

– Esborrar les dades velles costa molt: s'hauria de fer amb `DELETE` fila a fila, i l'espai no queda lliure fins que passa el `VACUUM`. Una base de sèries temporals esborra dies sencers de cop.

– Es repeteix informació: el nom del node i de la magnitud es guarden a cada fila.

A3. Compressió per columnes

En lloc de guardar les dades fila a fila (temps, valor, temps, valor…), es guarden juntes totes les marques de temps d'una sèrie i, a part, tots els valors. Així les dades que queden una al costat de l'altra són del mateix tipus i s'assemblen molt, i n'hi ha prou de guardar la diferència amb l'anterior.

Amb marques de temps regulars funciona especialment bé. Si les lectures arriben cada 5 s (1793014800, 1793014805, 1793014810, 1793014815), les diferències són sempre 5 i la diferència entre diferències (delta de delta) és 0, que es guarda amb un sol bit. Si totes les diferències són iguals, InfluxDB només guarda el primer valor, el pas i el nombre de punts.

Amb valors que canvien poc passa el mateix: si una lectura és igual que l'anterior (23,4 → 23,4), només cal un bit que vol dir «igual que abans», i si canvia poc (23,4 → 23,5) només es guarden els pocs bits que han canviat. El meu firmware arrodoneix a una dècima i hi ha molts valors repetits. Al final, un punt ocupa 1–3 bytes en lloc de les desenes de bytes que ocupa una fila.

A4. On cal una base relacional

Per a l'inventari de dispositius (exercici 5 de la P4): una taula `dispositius` amb `device_id` (clau primària, el mateix identificador que el tag d'InfluxDB) i `ubicacio`. Són poques files, les manté una persona des del dashboard de Node-RED, es modifiquen (alta, baixa, canvi d'ubicació) i necessiten regles que la base de dades faci complir (clau primària, valors no nuls, sense duplicats). Grafana la farà servir per triar el dispositiu pel nom de la ubicació.

La ubicació no es posa com a tag a InfluxDB perquè les lectures antigues no es poden modificar: si un node canviés de lloc, la seva sèrie quedaria partida en dues.

Bloc B · Model de dades a InfluxDB

B1. Tag i field

Un tag és una etiqueta de text que identifica la mesura (el dispositiu, la magnitud). Un field és el valor mesurat, que pot ser un número, un booleà o un text. Els tags estan indexats i els fields no.

Conseqüència: filtrar o agrupar per un tag és ràpid, però cada valor diferent d'un tag crea una sèrie nova i fa créixer l'índex, que InfluxDB 1.x guarda a la memòria. Per això un tag ha de tenir pocs valors possibles. Filtrar per un field obliga a llegir tots els punts del rang de temps i no s'hi pot agrupar, però és on es calculen les mitjanes, els màxims i els mínims.

B2. Sèrie i cardinalitat

Una sèrie és el conjunt de punts que tenen la mateixa measurement i els mateixos valors de tags, per exemple totes les lectures de temperatura del `node01`. La cardinalitat és el nombre de sèries diferents, i és el producte dels valors possibles de cada tag.

Amb 2 nodes, 3 magnituds i 1 ubicació: 2 × 3 × 1 = 6 sèries. El meu sistema té 5 dispositius × 2 magnituds = 10 sèries.

B3. La marca de temps com a tag

És un error greu perquè cada lectura té una marca de temps diferent: cada punt crearia una sèrie nova d'un sol punt, i la cardinalitat seria igual al nombre de punts escrits. Al cap d'un mes (30 dies), a 5 s:

- Sistema de B2 (6 sèries): 6 × 17.280 × 30 = 3.110.400 sèries.
- El meu sistema (10 sèries): 10 × 17.280 × 30 = 5.184.000 sèries.

Un Raspberry va bé per sota de 100.000 sèries. Amb milions, l'índex, que és a la memòria, creixeria fins a omplir la RAM. També es perdria la compressió, perquè cada sèrie tindria un sol punt i no hi hauria res amb què comparar, i les consultes anirien molt més lentes. A més, no cal: el temps ja és la dada principal de la base i InfluxDB ja hi pot filtrar (`WHERE time > ...`).

B4. Esquema de la telemetria

Element	Nom	Tipus	Valors i unitat
Measurement	`telemetria`	—	Totes les lectures dels sensors
Tag	`device_id`	text	`node01` (real) i `node02` a `node05` (simulats). El mateix identificador que a la taula dispositius de PostgreSQL
Tag	`magnitud`	text	`temp`, `hum`
Field	`valor`	float	°C (temp) o % HR (hum), amb una dècima
Temps	`time`	segons	La ts del missatge (hora NTP del node)

Cardinalitat: 5 × 2 = 10 sèries. Un punt queda així:

```
telemetria,device_id=node01,magnitud=temp valor=23.4 1793014800
```

És un esquema estret (un punt per magnitud), perquè la temperatura i la humitat arriben en temes i missatges separats, cadascun amb la seva hora. Els tags surten directament del tema MQTT. La unitat no es guarda perquè depèn de la magnitud, i la ubicació va a PostgreSQL.

A la P4 també hi haurà els resums horaris a la política freda (mateixos tags i els fields `valor`, `minim`, `maxim` i `mostres`) i una measurement `alarmes` (tags `device_id` i `magnitud`, fields `actiu` i `valor`).

B5. Volum en un any

- Mostres al dia d'una sèrie: 86.400 s / 5 s = 17.280.
- Mostres a l'any d'una sèrie: 17.280 × 365 = 6.307.200.
- Mostres a l'any del sistema: 6.307.200 × 10 sèries = 63.072.000.
- Mida a 10 bytes per punt: 63.072.000 × 10 = 630.720.000 bytes, uns 631 MB l'any.

Només el node real (2 sèries) serien 12.614.400 mostres i 126 MB.

Bloc C · Retenció i agregació

C1. Emmagatzematge calent i fred

L'emmagatzematge calent guarda les dades recents tal com arriben, amb tot el detall (una lectura cada 5 s), però només uns quants dies. El llegeixen el tauler de Grafana que mostra com està el sistema ara mateix i qui vol mirar què va passar fa poc (per exemple, dimarts a la nit).

L'emmagatzematge fred guarda les dades antigues resumides, per exemple un valor per hora, durant molt de temps. El llegeix qui vol veure com evoluciona el sistema durant setmanes o mesos, comparar un mes amb un altre o fer un informe.

Les dades recents es consulten molt i amb detall; de les antigues n'hi ha prou amb un resum.

C2. Polítiques de retenció

Política	Durada	Resolució	Què guarda	Punts	Mida (10 B)
`calenta` (per defecte)	7 dies	5 s	Cada lectura	10 × 17.280 × 7 = 1.209.600	~12 MB
`freda`	365 dies	1 hora	Mitjana, mínim, màxim i nombre de mostres de cada hora	10 × 24 × 365 = 87.600 (× 4 valors)	~3,5 MB

- 7 dies a la calenta: és prou per mirar amb tot el detall qualsevol problema de l'última setmana. Les dades de 5 s més antigues gairebé no es consulten.
- 1 any a la freda: permet veure com canvia la temperatura d'un mes a l'altre i entre l'estiu i l'hivern. Com que ocupa molt poc, es podria guardar més temps.
- 1 hora de resolució: és el que cal per fer gràfics de setmanes o mesos. Un mes són 720 punts per sèrie, en lloc dels 518.400 que hi hauria amb dades de 5 s.

La freda l'omplirà una consulta contínua que s'executa cada hora i calcula el resum a partir de la calenta.

C3. Què es perd amb el downsampling

Passar d'una sèrie de 10 s a mitjanes horàries vol dir que 360 mostres es converteixen en un sol número. Es perd:

- els valors màxims i mínims, perquè la mitjana els aplana;
- l'hora exacta i la durada del que ha passat dins de l'hora;
- com de variable ha estat la temperatura dins de l'hora;
- els forats sense dades: si el node no envia res durant 40 minuts, la mitjana es calcula amb els 20 minuts que queden i sembla una hora normal.

Exemple: si la temperatura està 55 minuts a 24 °C i 5 minuts a 32 °C, la mitjana de l'hora és (55 × 24 + 5 × 32) / 60 = 24,7 °C. Amb només la mitjana ja no es pot respondre «ahir la temperatura va superar els 30 °C en algun moment?».

C4. Guardar també el mínim, el màxim i el recompte

Amb el màxim es torna a poder respondre la pregunta anterior: el màxim d'aquella hora és 32 °C, així que sí que va superar els 30 °C, i se sap en quina hora va ser.

El recompte, a més, diu si hi ha hagut forats. Amb mostres cada 10 s, una hora completa en té 360 (cada 5 s, 720). Si n'hi ha menys, el node va deixar d'enviar dades una estona i aquella mitjana és menys fiable.

El que no es recupera és el minut exacte en què va passar i quant va durar.

C5. Retenció i disc del Raspberry

- Calenta (7 dies): 1.209.600 punts × 10 B ≈ 12 MB (fins a uns 14 MB, perquè InfluxDB esborra per blocs d'un dia i hi pot haver fins a 8 dies).
- Freda (1 any): 87.600 punts × 4 valors × 10 B ≈ 3,5 MB.
- Alarmes: menys d'1 MB.

En total, uns 17 MB com a màxim. La mida queda fixa: la calenta s'omple la primera setmana i després es manté (cada dia entra un dia nou i se n'esborra un de vell), i la freda creix el primer any i després també es manté. És el 0,2 % dels 9,1 GB lliures.

Sense retenció, la base creixeria 631 MB cada any sense parar. Amb un sistema tan petit, la targeta trigaria uns 14 anys a omplir-se, però la base seria cada cop més gran, les consultes llargues més lentes i la targeta es gastaria més. Amb més nodes o un període més curt, el problema arribaria molt abans.

Preparació pràctica

Instal·lació d'InfluxDB i Grafana

Tots dos s'han instal·lat al Raspberry amb `apt`, des dels repositoris oficials d'InfluxData i de Grafana Labs. Les ordres són a `docs/pre4/instalacio.md`.

- InfluxDB 1.8.10, port 8086, servei `influxdb`. El Raspberry porta Raspbian de 32 bits (`armhf`) i, per a aquest sistema, el repositori només té InfluxDB fins a la 1.8.10. Té tot el que demana la pràctica: InfluxQL, polítiques de retenció i consultes contínues.
- Grafana 13.3.0, port 3000, servei `grafana-server`.

Tots dos serveis estan habilitats amb `systemctl enable`. Per comprovar que arrenquen sols, s'ha reiniciat el Raspberry (`sudo reboot`). Després, `uptime -s` mostrava l'hora del reinici, InfluxDB, Grafana, Mosquitto i Node-RED sortien com a `enabled` i `active`, i InfluxDB responia al `/ping` amb el codi 204.

Accés autenticat: a InfluxDB s'ha creat un usuari administrador i s'ha activat `auth-enabled = true`; sense usuari i contrasenya no deixa fer res. A Grafana s'ha canviat la contrasenya de l'administrador i s'han desactivat el registre d'usuaris nous i l'accés anònim.

Consulta InfluxQL

Mitjana de cada hora de la temperatura de les últimes 24 hores (sense executar, perquè la base de dades i les polítiques es creen a la P4):

```
SELECT mean("valor") AS "mitjana"
FROM "iot"."calenta"."telemetria"
WHERE "magnitud" = 'temp'
  AND "device_id" = 'node01'
  AND time > now() - 24h
GROUP BY time(1h) fill(none) tz('Europe/Madrid')
```

Es filtra per tags (`magnitud` i `device_id`) i per temps, que són els filtres ràpids. `GROUP BY time(1h)` fa un grup per hora, `fill(none)` no mostra les hores sense dades i `tz` posa l'hora d'aquí.

Espai lliure a la targeta

`df -h /` abans d'instal·lar: targeta de 14 GB, 4,1 GB usats i 9,1 GB lliures (32 % ocupat). La instal·lació d'InfluxDB i Grafana n'ocupa uns 1,5 GB. Queda molt per sobre dels 2 GB que demana la pràctica.

Flux de la P3

Amb `mosquitto_sub` s'ha comprovat que el `node01` continua publicant: `status` surt com a `online` i arriben la temperatura i la humitat cada 5 s.

Fitxers al repositori

A `docs/pre4/` hi ha:

- `instalacio.md`: les ordres d'instal·lació, configuració i comprovació.
- `influxdb.conf`: les línies actives de `/etc/influxdb/influxdb.conf`, comentades.
- `grafana.ini`: les línies activades a `/etc/grafana/grafana.ini`, comentades.
- `consulta.influxql`: la consulta d'aquest apartat.

Les contrasenyes no s'hi guarden.

La targeta microSD

InfluxDB escriu a la targeta cada vegada que arriba una dada, i una targeta microSD aguanta un nombre limitat d'escriptures. En un sistema real, que ha d'estar mesos engegat, faria això:

- Enviar dades menys sovint: una temperatura ambient no canvia en 5 s; amb 30 s n'hi hauria prou i s'escriuria 6 vegades menys.
- Escriure per lots: juntar unes quantes lectures i escriure-les de cop.
- Guardar els registres del sistema a la RAM en lloc de la targeta.
- Fer servir un disc SSD per USB, o una targeta pensada per a moltes escriptures.
- Fer còpies de seguretat de les dades i de la configuració, i comprovar que es poden recuperar.